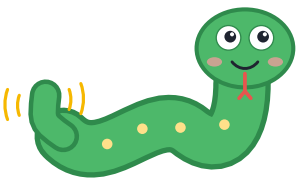
Lecture 2

Part 1: Intro and Python Basics

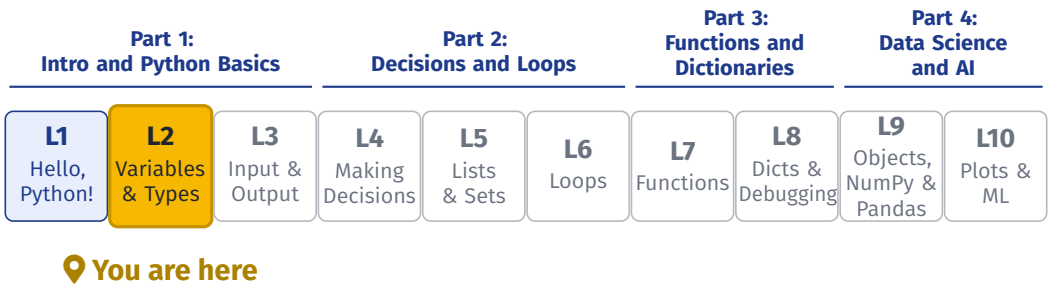
Variables & Types

Teaching the computer to remember, and Python as a super calculator



Dror Jacoby
Tel Aviv University

Where we are


Part 1:
Intro and Python Basics
Part 2:
Decisions and Loops
Part 3:
Functions and
Dictionaries
Part 4:
Data Science
and AI
L1
Hello,
Python!
L2
Variables
& Types
L3
Input &
Output
L4
Making
Decisions
L5
Lists
& Sets
L6
Loops
L7
Functions
L8
Dicts &
Debugging
L9
Objects,
NumPy &
Pandas
L10
Plots &
ML
You are here

Who remembers phone numbers anymore?

- You don’t type 054-… to call Mom

Who remembers phone numbers anymore?

- You don’t type 054-… to call Mom
- You tap **“Mom”**. The phone remembers the number.
- A **name** that holds a **value**

Who remembers phone numbers anymore?

- You don’t type 054-… to call Mom
- You tap **“Mom”**. The phone remembers the number.
- A **name** that holds a **value**
- Today: teach Python to remember things too





Today's plan

◎ By the end of today you can...

- Store values in variables and change them
- Tell numbers, text and True/False apart
- Use Python as a calculator for everyday problems



Today's outline

1 **Variables: labeled boxes**
Store a value under a name, update it, choose good names

2 **Types: what kind of value?**
`int`, `float`, `str`, `bool`, and asking Python with `type()`

3 **Python as a calculator**
`+ - * /`, `//`, `%`, `**`, order of operations, math on text



Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- Every program remembers things: a score, a price, a name
- The type decides what you can do: add numbers, glue text
- Most beginner bugs are type bugs. Today you learn to spot them

Where you will see it

- Your bank app: balance (a `float`)
- Instagram: likes (`int`), username (`str`)
- Logged in or not? (`bool`)





Topic 1

Variables: labeled boxes

Lecture 2 • Variables & Types



TODAY

1. **Variables: labeled boxes**
2. Types: what kind of value?
3. Python as a calculator

A variable is a labeled box

- The **label** is the name: `price`
- **Inside** is the value: `12`
- Python remembers it until you change it



Your first variable

```
price = 12
print(price)
```

Your first variable

```
price = 12
print(price)
```

OUTPUT

```
12
```

Your first variable

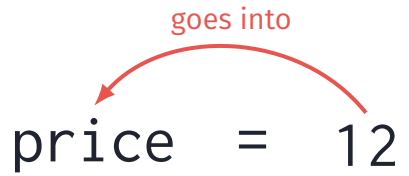
```
price = 12
print(price)
```

OUTPUT

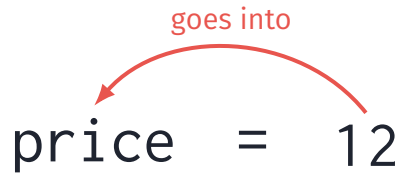
```
12
```

- Line 1 puts `12` into a box called `price`
- Line 2: no quotes, so Python opens the box

= means “store”, not “equals”

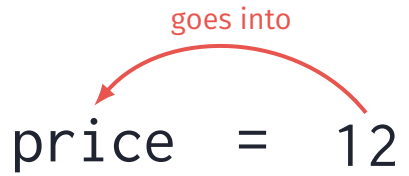


= means “store”, not “equals”



- Read it as: “**price gets 12**”
- The value on the right goes into the box on the left

= means “store”, not “equals”



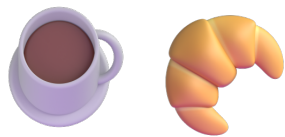
- Read it as: “**price gets 12**”
- The value on the right goes into the box on the left

⚠ Not like math class

In math, = means “is the same as”. In Python it means “put into the box”.

Variables can work together

```
coffee = 14
croissant = 12
total = coffee + croissant
print(total)
```



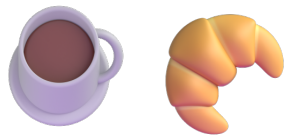
Python opens both boxes, adds, and stores the result.

Variables can work together

```
coffee = 14
croissant = 12
total = coffee + croissant
print(total)
```

OUTPUT

```
26
```



Python opens both boxes, adds, and stores the result.

A box holds one value at a time

```
num_likes = 10
print(num_likes)
num_likes = 11
print(num_likes)
```

A box holds one value at a time

```
num_likes = 10
print(num_likes)
num_likes = 11
print(num_likes)
```

OUTPUT

```
10
11
```

A box holds one value at a time

```
num_likes = 10
print(num_likes)
num_likes = 11
print(num_likes)
```

```
OUTPUT
10
11
```



The old value is thrown away.

Update a box using its own value

```
score = 0
score = score + 10
score = score + 5
print(score)
```



Update a box using its own value

```
score = 0
score = score + 10
score = score + 5
print(score)
```



Right side first: `0 + 10` is `10`, then store it back in `score`.

Update a box using its own value

```
score = 0
score = score + 10
score = score + 5
print(score)
```



OUTPUT

```
15
```

Right side first: `0 + 10` is `10`, then store it back in `score`.

Naming rules

Allowed

- Letters, digits, underscore _
- `price`, `num_students`, `exam2`

⚠ Not allowed

- Start with a digit: `2nd_place`
- Spaces: `my score`
- Python words: `class`, `if`

Naming rules

Allowed

- Letters, digits, underscore `_`
- `price`, `num_students`, `exam2`

⚠ Not allowed

- Start with a digit: `2nd_place`
- Spaces: `my score`
- Python words: `class`, `if`

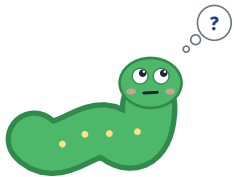
Capitals matter

`price` and `Price` are two different boxes.

Good names tell a story

Hard to read

```
x = 30
y = 4
```



Write for “future you”
at 2 a.m. before a deadline.

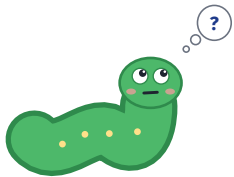
Good names tell a story

Hard to read

```
x = 30
y = 4
```

Easy to read

```
num_students = 30
num_groups = 4
```



Write for “future you”
at 2 a.m. before a deadline.

Try it!

Try it in Colab notebooks/L02/L02_01_variables.ipynb

1. Store your age in a variable called `age`
2. Store the price of your favorite snack in `snack_price`
3. Buy 3 snacks: `total = 3 * snack_price`, then print it



Predict the output

Predict the output!

```
a = 5
b = a
a = 10
print(b)
```



Predict the output

Predict the output!

```
a = 5
b = a
a = 10
print(b)
```

Answer

5

`b = a` copies the value *at that moment*. Changing `a` later does not touch `b`.



Topic 2

Types: what kind of value?

Lecture 2 • Variables & Types



TODAY

1 Variables: labeled boxes

2 Types: what kind of value?

3 Python as a calculator

Values come in different kinds

	Kind	Python name	Example	In real life
🔢	Whole number	`int`	`30`	students in class
🪙	Decimal number	`float`	`12.5`	price of a coffee
💬	Text	`str`	`"Dana"`	a name, a message
💡	True or False	`bool`	`True`	is it raining?

Values come in different kinds

	Kind	Python name	Example	In real life
🔢	Whole number	`int`	`30`	students in class
🪙	Decimal number	`float`	`12.5`	price of a coffee
💬	Text	`str`	`"Dana"`	a name, a message
💡	True or False	`bool`	`True`	is it raining?

The type decides what you can do with a value.

Numbers: `int` and `float`

```
num_students = 30      # int
temperature = -3       # int
price = 12.5           # float
print(price)
```

OUTPUT

```
12.5
```

- **int**: whole numbers, also negative
- **float**: has a decimal point



Text: `str` (string)

```
city = "Tel Aviv"
greeting = 'Shalom!'
quote = "Dana's phone"
print(city)
print(greeting)
print(quote)
```

Text: `str` (string)

```
city = "Tel Aviv"
greeting = 'Shalom!'
quote = "Dana's phone"
print(city)
print(greeting)
print(quote)
```

OUTPUT

```
Tel Aviv
Shalom!
Dana's phone
```

Text: `str` (string)

```
city = "Tel Aviv"
greeting = 'Shalom!'
quote = "Dana's phone"
print(city)
print(greeting)
print(quote)
```

OUTPUT

```
Tel Aviv
Shalom!
Dana's phone
```

- **String** = text in quotes, `"double"` or `'single'`
- Text has an apostrophe? Use double quotes.

True or False: `bool`

```
is_raining = True
has_exam = False
print(is_raining)
```

OUTPUT

```
True
```

Like a light switch: only **on** or **off**.
Capital `T` and `F`, no quotes!



Named after George Boole,
a 19th-century mathematician

Ask Python: `type()`

```
print(type(30))
print(type(12.5))
print(type("Dana"))
print(type(True))
```

Ask Python: `type()`

```
print(type(30))
print(type(12.5))
print(type("Dana"))
print(type(True))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Ask Python: `type()`

```
print(type(30))
print(type(12.5))
print(type("Dana"))
print(type(True))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

`type()` tells you the kind of any value. Read **`class`** as “kind”.

Quotes change everything

```
print(5 + 5)
print("5" + "5")
print(type("5"))
```



Quotes change everything

```
print(5 + 5)
print("5" + "5")
print(type("5"))
```

OUTPUT

```
10
55
<class 'str'>
```



Quotes change everything

```
print(5 + 5)
print("5" + "5")
print(type("5"))
```

OUTPUT

```
10
55
<class 'str'>
```

"5" is not a number. It is the *character* 5, like on a T-shirt.



One letter is just a short string

```
letter = "A"
print(type(letter))
print(len(letter))
print(len("pizza"))
```



One letter is just a short string

```
letter = "A"
print(type(letter))
print(len(letter))
print(len("pizza"))
```

OUTPUT

```
<class 'str'>
1
5
```



One letter is just a short string

```
letter = "A"
print(type(letter))
print(len(letter))
print(len("pizza"))
```

OUTPUT

```
<class 'str'>
1
5
```

- No special “letter” type
- A letter is a string of length 1
- `len()` counts the characters



Try it!

Try it in Colab notebooks/L02/L02_02_types.ipynb

1. Make one variable of each type: `int`, `float`, `str`, `bool`
2. Check each one with `type()`
3. Guess first: what is `type("3.5")`? Then run it.



Predict the output

Predict the output!

```
print(type(7.0))
print(type("7"))
print(type(7))
```



Predict the output

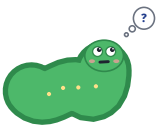
Predict the output!

```
print(type(7.0))
print(type("7"))
print(type(7))
```

Answer

```
<class 'float'>
<class 'str'>
<class 'int'>
```

A dot makes a float. Quotes make a string.



Topic 3

Python as a calculator

Lecture 2 • Variables & Types



TODAY

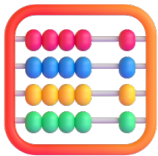
1 Variables: labeled boxes

2 Types: what kind of value?

3 Python as a calculator

The math symbols

Symbol	Meaning	Example
`+ -`	plus, minus	`7 - 3` → 4
`*`	times	`7 * 3` → 21
`/`	divide	`7 / 2` → 3.5
`//`	divide, drop the rest	`7 // 2` → 3
`%`	the leftover (remainder)	`7 % 2` → 1
`**`	power	`7 ** 2` → 49



/ always gives a float

```
print(10 / 2)
print(9 / 4)
```

/ always gives a float

```
print(10 / 2)
print(9 / 4)
```

OUTPUT

```
5.0
2.25
```

/ always gives a float

```
print(10 / 2)
print(9 / 4)
```

OUTPUT

```
5.0
2.25
```

- Even when it divides evenly: `5.0`, not `5`
- Python plays it safe: the answer might need decimals

Split the pizza: // and %

```
slices = 8
friends = 3
print(slices // friends)
print(slices % friends)
```



Split the pizza: `//` and `%`

```
slices = 8
friends = 3
print(slices // friends)
print(slices % friends)
```



OUTPUT

```
2
2
```

Split the pizza: `//` and `%`

```
slices = 8
friends = 3
print(slices // friends)
print(slices % friends)
```

OUTPUT

```
2
2
```



- `//`: each friend gets **2** slices
- `%`: **2** slices are left over

How long is the movie?

```
movie_minutes = 135
print(movie_minutes // 60)  # hours
print(movie_minutes % 60)   # minutes
```



How long is the movie?

```
movie_minutes = 135
print(movie_minutes // 60)  # hours
print(movie_minutes % 60)   # minutes
```



OUTPUT

```
2
15
```

How long is the movie?

```
movie_minutes = 135
print(movie_minutes // 60)  # hours
print(movie_minutes % 60)   # minutes
```

OUTPUT

```
2
15
```



135 minutes = **2 h 15 min**

Who goes first?

1. Parentheses `( )`
2. Power `**`
3. `* / // %`
4. `+ -`

Same as in school math.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

Who goes first?

1. Parentheses `( )`
2. Power `**`
3. `* / // %`
4. `+ -`

Same as in school math.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

OUTPUT

```
14
20
```

Who goes first?

1. Parentheses `( )`
2. Power `**`
3. `* / // %`
4. `+ -`

Same as in school math.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

OUTPUT

```
14
20
```

Not sure? Add parentheses.

Average of three exam grades

```
print(85 + 92 + 78 / 3)
print((85 + 92 + 78) / 3)
```



Average of three exam grades

```
print(85 + 92 + 78 / 3)
print((85 + 92 + 78) / 3)
```

OUTPUT

```
203.0
85.0
```



Average of three exam grades

```
print(85 + 92 + 78 / 3)
print((85 + 92 + 78) / 3)
```

OUTPUT

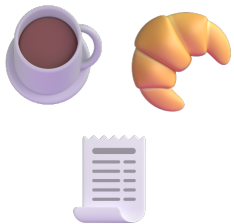
```
203.0
85.0
```



Line 1 divides only the 78. Oops!

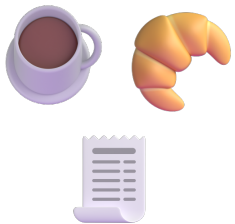
Café bill for three friends

```
coffee = 14
croissant = 12.5
num_friends = 3
bill = num_friends * (coffee + croissant)
print(bill)
```



Café bill for three friends

```
coffee = 14
croissant = 12.5
num_friends = 3
bill = num_friends * (coffee + croissant)
print(bill)
```

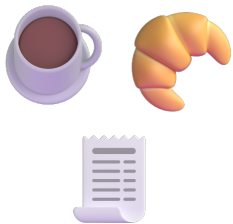


OUTPUT

```
79.5
```

Café bill for three friends

```
coffee = 14
croissant = 12.5
num_friends = 3
bill = num_friends * (coffee + croissant)
print(bill)
```



OUTPUT

```
79.5
```

Change `num_friends` to 5 and run again. No retyping!

Power: **

```
print(2 ** 10)
side = 4
print(side ** 2)
```

Power: **

```
print(2 ** 10)
side = 4
print(side ** 2)
```

OUTPUT

```
1024
16
```

Power: **

```
print(2 ** 10)
side = 4
print(side ** 2)
```

OUTPUT

```
1024
16
```

- `2 ** 10`: double 1 ten times
- `side ** 2`: area of a 4 by 4 square

Strings can do + and * too

```
first = "Tel"
second = "Aviv"
print(first + " " + second)
print("ha" * 3)
print("-" * 20)
```



Strings can do + and * too

```python
first = "Tel"
second = "Aviv"
print(first + " " + second)
print("ha" * 3)
print("-" * 20)
```

OUTPUT

```
Tel Aviv
hahaha
--------------------
```



Strings can do + and * too

```
first = "Tel"
second = "Aviv"
print(first + " " + second)
print("ha" * 3)
print("-" * 20)
```

OUTPUT

```
Tel Aviv
hahaha
--------------------
```

- + glues strings together
- * repeats a string



Try it!

Try it in Colab notebooks/L02/L02_03_calculator.ipynb

1. Hours in a year: `365 * 24`
2. 100 cookies, 7 friends: how many each? How many left?
3. Your last 3 grades in variables. Print the average.





Predict the output

Predict the output!

```
print(10 / 4)
print(10 // 4)
print(10 % 4)
```



Predict the output

Predict the output!

```
print(10 / 4)
print(10 // 4)
print(10 % 4)
```

Answer

```
2.5
2
2
```

10 into groups of 4: 2 full groups, 2 left over.



Bug Zoo: mixing text and numbers

BUG ZOO

```
age = 20
print("Age: " + age)
```

ERROR

```
TypeError: can only concatenate
str (not "int") to str
```

Text + number = ?

“Concatenate” means “glue”.
You can glue text only to text.

Fix for now: `"Age: " + "20"`

Next lecture: a better fix.

Bug Zoo: a box that does not exist

BUG ZOO

```
total = 26
print(totl)
```

ERROR

```
NameError: name 'totl' is not
defined. Did you mean: 'total'?
```

Fix: a typo! Write `total`

```
price = 12
print(Price)
```

ERROR

```
NameError: name 'Price' is not
defined. Did you mean: 'price'?
```

Fix: capitals matter. Write `price`



Python even suggests the fix. Read the **last line**!

Bug Zoo: illegal names

BUG ZOO

```
2nd_place = "Dana"
```

ERROR

```
SyntaxError: invalid decimal
literal
```

Fix: start with a letter: `second_place`

```
my score = 90
```

ERROR

```
SyntaxError: invalid syntax
```

Fix: no spaces: `my_score`



"Invalid decimal literal": Python saw a `2` and expected a number.

AI Corner: explain my variables

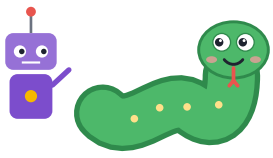
AI CORNER

A good prompt

"I am a Python beginner. Explain line by line what happens to each variable in this code. Do not rewrite it." + your code

Double-check

Predict the final values yourself first. Then run it: who was right, you or the AI?



Recap

✔ Today you learned

- A variable is a labeled box. = means “store”.
- Four types: `int`, `float`, `str`, `bool`. Check with `type()`.
- `+ - * / // % **` work like a calculator. Text can do + and *.



Next time: programs that talk with you

- Ask the user a question with `input()`
- Turn text into numbers and back
- Pretty output, and your first **`if`**



3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md